Digital Integrated Circuit Design

Lecture-4: Additions and Subtractions

Chih-Feng Wu (吳志峰), PhD.

Department of Electronics Engineering, Chang Gung University,
Taoyuan, Taiwan (ROC)

Mar. 28, 2025

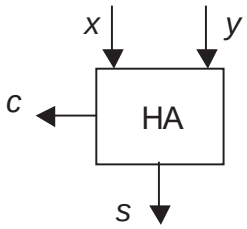
Outline

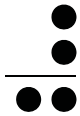
- ❑ **Half- and Full-Adder**
- ❑ **Ripple Carry Adder**
- ❑ **Carry-Lookahead Adder**
- ❑ **Carry-Skip Adder**
- ❑ **Carry-Select Adder**
- ❑ **Conditional-Sum Adder**
- ❑ **Modular Two-Operand Adders**

Half- and Full-Adder

- Half adder (HA)

Inputs		Outputs	
x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



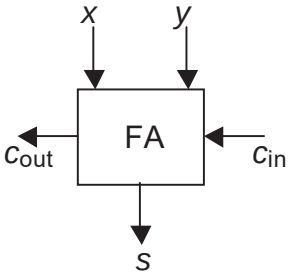


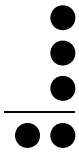
$s = x \oplus y$

$c = xy$

- Full adder (FA)

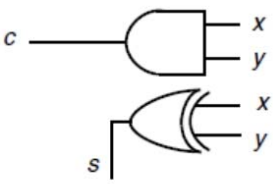
Inputs			Outputs	
x	y	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



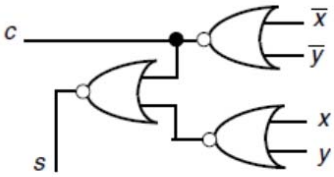


Half- and Full-Adder (cont'd)

- 3 implementations of a HA
  - (a)

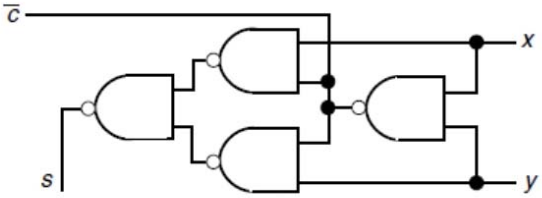


(a) AND/XOR HA.



(b) NOR-gate HA.

  - (b)



(c) NAND-gate HA with complemented carry.

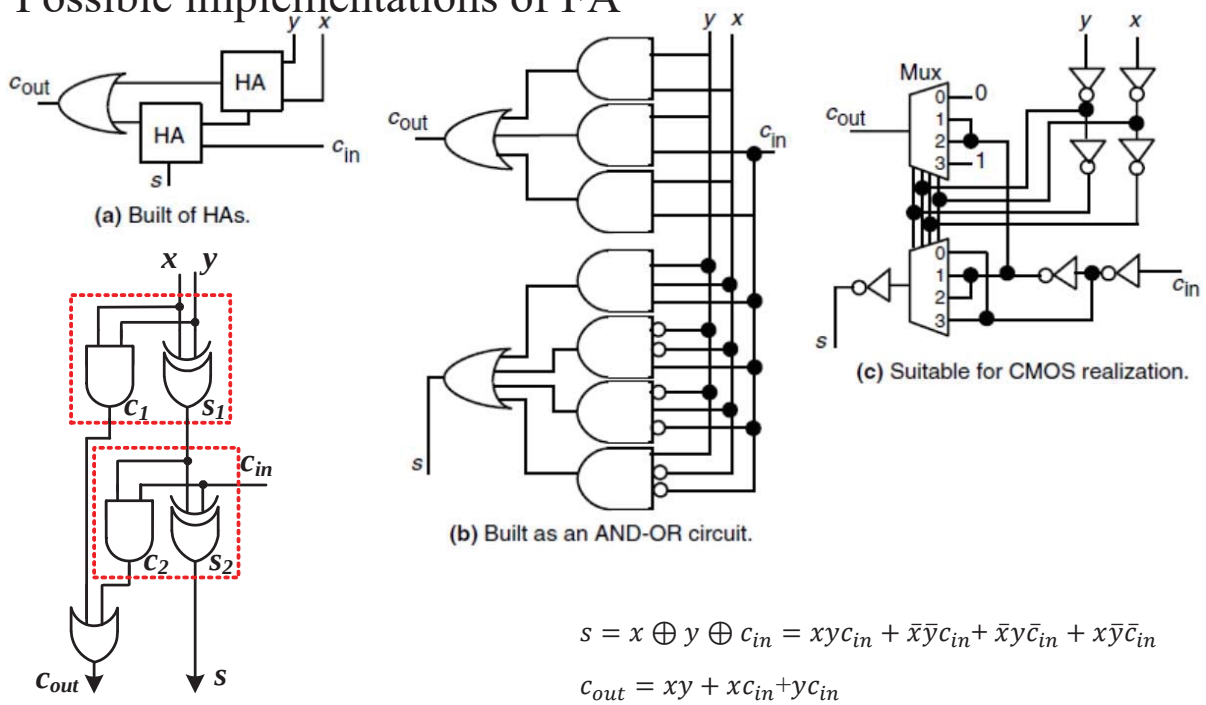
  - (c)

Discussion of Delay Time

- Delay time for various logic gates
  - $t_{inv}, t_{nand}, t_{nor}$
  - t_{and}, t_{or}, t_{xor}

Half- and Full-Adder (cont'd)

- Possible implementations of FA

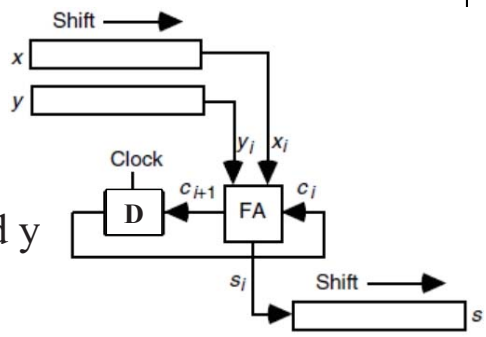


$$s = x \oplus y \oplus c_{in} = xyc_{in} + \bar{x}\bar{y}c_{in} + \bar{x}y\bar{c}_{in} + x\bar{y}\bar{c}_{in}$$

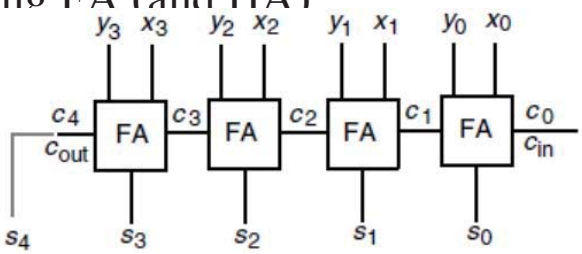
$$c_{out} = xy + xc_{in} + yc_{in}$$

Bit Aerial Adder and Ripple Carry Adder

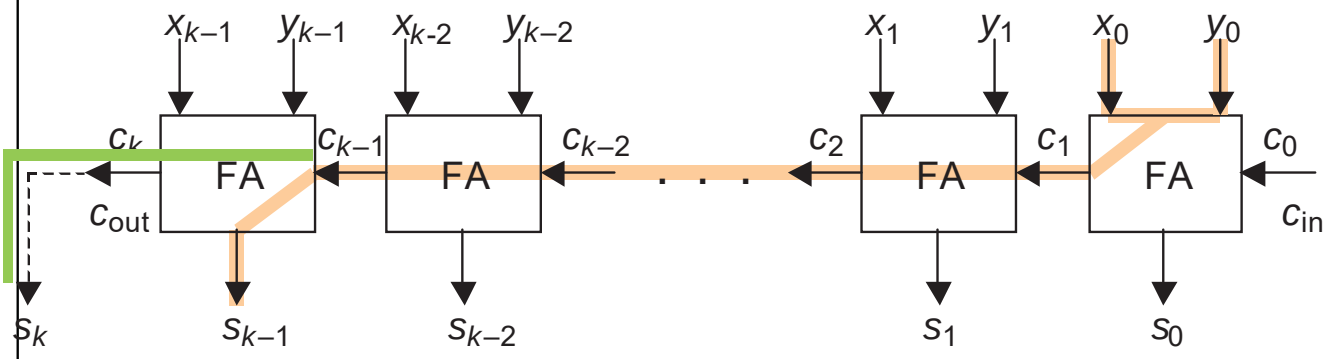
- Bit serial adder using FA
  - t_d of N-bit bit serial adder
  - D: the carry D-FF
  - Time complexity O(·) for ***N***-bit of x and y
    - The no. of clock cycle = _____
    - Clock rate (frequency) ∝ _____



- Ripple carry adder (RCA) using FA (and HA)
  - t_d (4-bit)



Critical Path for a k-bit Ripple Carry Adder



- Delay time (***k***-bit RCA)

$T_{RCA} = T_{FA}(x,y \rightarrow c_{out}) + (k-2) \times T_{FA}(c_{in} \rightarrow cout) + T_{FA}(c_{in} \rightarrow s)$; // Build AND-OR FA

$T_{RCA} = T_{FA}(x,y \rightarrow c_{out}) + (k-1) \times T_{FA}(c_{in} \rightarrow cout)$; // Build **HAs**

$T_{RCA} \approx \boldsymbol{k} T_{FA}(c_{in} \rightarrow cout)$; // Build **HAs**

- Time complexity= $O(k)$

Conditions and Exceptions

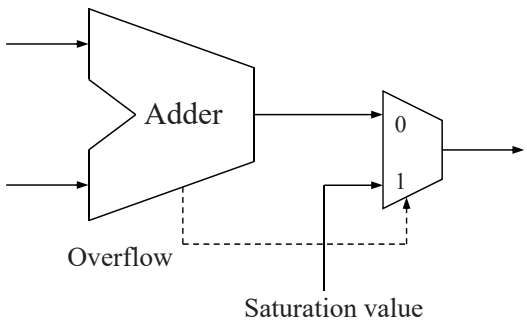
- 4 flags indicating the status of an arithmetic logic unit
  - c_{out}: Generation for carry-out of 1
  - Overflow: Incorrect output sum
  - Negative: Negative result of addition
  - Zero: Zero result of addition
- Overflow detection
  - $OV_{2's-complement} = x_{k-1}y_{k-1}\bar{s}_{k-1} + \bar{x}_{k-1}\bar{y}_{k-1}s_{k-1}$
  - $OV_{2's-complement} = c_k \oplus c_{k-1}$

Conditions and Exceptions (cont'd)

- Saturation detection and realization
  - Too large of a magnitude of an adder result
    - Outcome: wrap around result
    - Solution: Provide the most positive or the most negative value (Saturating the results of the adder)
  - Example – In 8-bit 2's-complement format, we have:

    120 + 26 → 18 (wraparound)

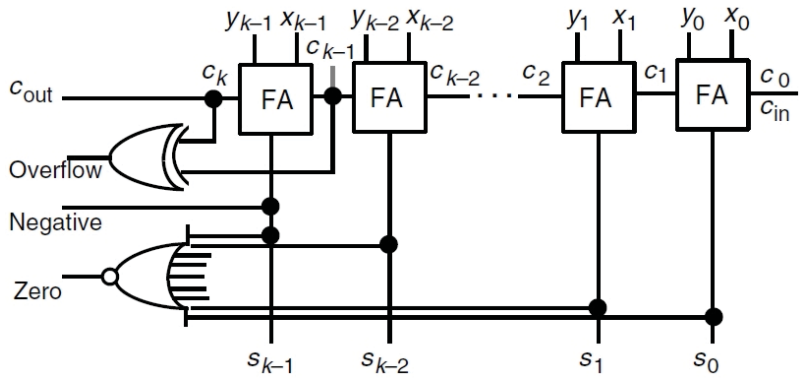
    $120 +_{sat} 26$ → 127 (saturating)



- Designing saturating adders
  - Unsigned (quite easy)
  - Signed (only slightly harder)

Conditions and Exceptions (cont'd)

- 2's complement adder with provisions for detecting conditions and exceptions



  - Overflow detection
    - $OV_{2's-complement} = x_{k-1}y_{k-1}\bar{s}_{k-1} + \bar{x}_{k-1}\bar{y}_{k-1}s_{k-1}$
    - $OV_{2's-complement} = c_k \oplus c_{k-1}$

Manchester Carry Chains and Adders

- Fast addition for two operands
  - Low-latency carry network
  - $s_i = x_i \oplus y_i \oplus c_i$ for full-adder

- Consider the carry propagation and the design of a carry network
  - Half-adder (addition for two operands without carry-in)
    - $s_i = x_i \oplus y_i = \bar{x}_i y_i + x_i \bar{y}_i$
    - $c_{out} = x_i y_i$

x_i	y_i	c_{out}	s_i	a_i
0	0	0	0	1
0	1	0	1	0
1	0	0	1	0
1	1	1	0	0

Manchester Carry Chains and Adders (cont'd)

- Generate, propagate, annihilate/absorb and transfer signals
  - $g_i = x_i y_i$ (carry generation)
  - $p_i = x_i \oplus y_i$ (carry propagation)
  - $a_i = \bar{x}_i \bar{y}_i = \overline{x_i + y_i}$ (carry annihilation)

- Define a **transfer signal** t_i corresponding to an event that the carry-out will be 1, given the carry-in is 1.
  - $t_i = g_i + p_i = \bar{a}_i = x_i + y_i$ (carry non-annihilation)
- In general, g_i, p_i, and a_i can be reformulated as

$$g_i = 1, \quad iff \;\; x_i + y_i \geq 2$$
$$p_i = 1, \quad iff \;\; x_i + y_i = 1$$
$$a_i = 1, \quad iff \;\; x_i + y_i < 1$$

g_i	p_i	Carry
0	0	Annihilated (absorb)
0	1	Propagated
1	0	Generated
1	1	Impossible

Manchester Carry Chains and Adders (cont'd)

- The **carry recurrence** for FA can be derived as follows:
  - $c_{i+1} = x_i y_i + c_i(x_i \oplus y_i)$ → XOR
    $= g_i + c_i p_i$
    $g_i = x_i y_i$ and $p_i = x_i \oplus y_i$
  - $c_{i+1} = g_i + c_i p_i$
    $= g_i(1 + c_i) + c_i p_i$
    $= g_i + c_i(g_i + p_i)$
    $= g_i + c_i t_i$
    $= x_i y_i + c_i(x_i + y_i)$ → OR
  - $c_{out} = xy + xc_{in} + yc_{in}$ → FA result
  - $c_{i+1} = x_i y_i + x_i c_i + y_i c_i$ → the ith FA of an ripple adder
    $= x_i y_i + c_i(x_i + y_i)$

x_i	y_i	c_i	c_{out}	s_i	g_i	p_i	t_i
0	0	0	**0**	0	0	0	0
0	0	1	**0**	1	0	0	0
0	**1**	0	**0**	1	0	1	1
0	**1**	1	**1**	0	0	1	**1**
1	**0**	0	**0**	1	0	1	1
1	**0**	1	**1**	0	0	1	**1**
1	**1**	0	**1**	0	**1**	0	0
1	**1**	1	**1**	1	**1**	0	0

Manchester Carry Chains and Adders (cont'd)

- Case-1: carry generate
  - If $a_0 b_0 = 1$ then $c_1 = 1$.
- Case-2: carry propagation
  - If $a_0 \oplus b_0 = 1$ then $c_1 = 1$ with $c_0 = 1$.
  - If $a_0 + b_0 = 1$ then $c_1 = 1$ with $c_0 = 1$.

$$
\begin{array}{ccccc}
c_4 & c_3 & c_2 & c_1 & c_0 \\
 & a_3 & a_2 & a_1 & a_0 \\
+ & b_3 & b_2 & b_1 & b_0 \\
\hline
c_{out} & s_3 & s_2 & s_1 & s_0
\end{array}
$$

Case-1:

$$
\begin{array}{cc}
c_{in} = 0 & c_{in} = 1 \\
1 \quad 0 & 1 \quad 1 \\
1 & 1 \\
+ \quad 1 & + \quad 1 \\
\hline
0 & 1
\end{array}
$$

Case-2:

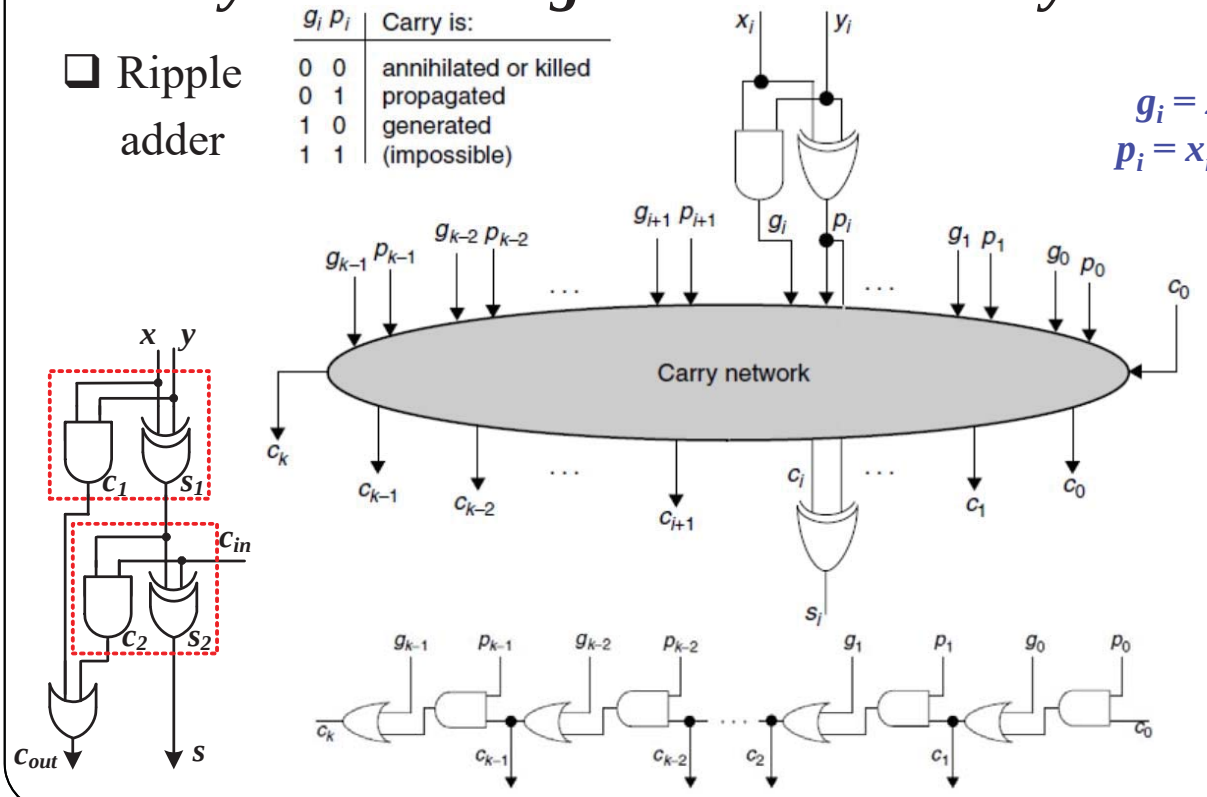
$$
\begin{array}{cc}
c_{in} = 1 & c_{in} = 1 \\
1 \quad 1 & 1 \quad 1 \\
0 & 1 \\
+ \quad 1 & + \quad 0 \\
\hline
0 & 0
\end{array}
$$

Binary Adder using Manchester Carry Chains

- Ripple adder

g_i p_i	Carry is:
0 0	annihilated or killed
0 1	propagated
1 0	generated
1 1	(impossible)

$g_i = x_i y_i$
$p_i = x_i \oplus y_i$



Unrolling The Carry Recurrence

- Generate, propagate, annihilate/absorb and transfer signals
  - $g_i = x_i y_i$ (carry generation)
  - $p_i = x_i \oplus y_i$ (carry propagation)
  - $a_i = \bar{x}_i \bar{y}_i = \overline{x_i + y_i}$ (carry annihilation)

x_i	y_i	c_{out}	s_i	a_i
0	0	0	0	1
0	1	0	1	0
1	0	0	1	0
1	1	1	0	0

$$g_i = 1, \quad iff \;\; x_i + y_i \geq 2$$
$$p_i = 1, \quad iff \;\; x_i + y_i = 1$$
$$a_i = 1, \quad iff \;\; x_i + y_i < 1$$

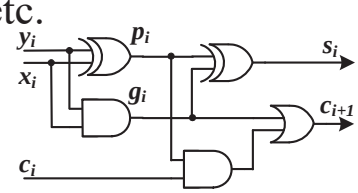
  - $t_i = g_i + p_i = \bar{a}_i = x_i + y_i$ (carry non-annihilation)
- Carry recurrence
  - $c_{i+1} = g_i + c_i p_i = g_i + c_i t_i$
  - The carry propagation from *i*th- to (*i*+*1*)th-stage

Unrolling The Carry Recurrence (cont'd)

- Unrolling carry recurrence
  - $c_{i+1} = g_i + c_i p_i;$ $\qquad c_i = g_{i-1} + c_{i-1} p_{i-1}$

$$\begin{aligned} &= g_i + (g_{i-1} + c_{i-1} p_{i-1}) p_i \\ &= g_i + g_{i-1} p_i + c_{i-1} p_{i-1} p_i \\ &= g_i + g_{i-1} p_i + (g_{i-2} + c_{i-2} p_{i-2}) p_{i-1} p_i \\ &= g_i + g_{i-1} p_i + g_{i-2} p_{i-1} p_i + c_{i-2} p_{i-2} p_{i-1} p_i = \cdots \end{aligned}$$

- Simple interpretation
  - Carry enters into ***i+1*** iff a carry is generated in ***i*** (g_i), or a carry generated in ***i-1*** is propagated by ***i*** ($g_{i-1} p_i$), a carry generated in ***i-2*** is propagated at ***i-1*** and ***i*** ($g_{i-2} p_{i-1} p_i$), etc.

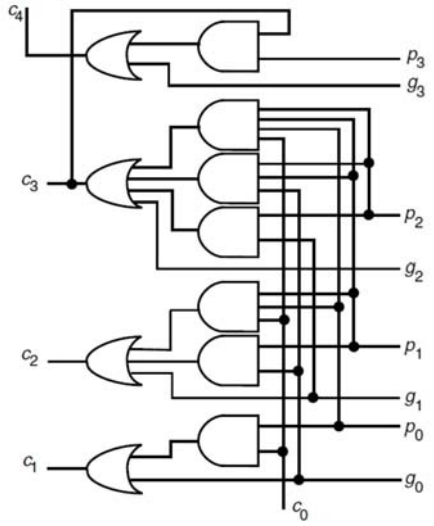


4-Bit CLA

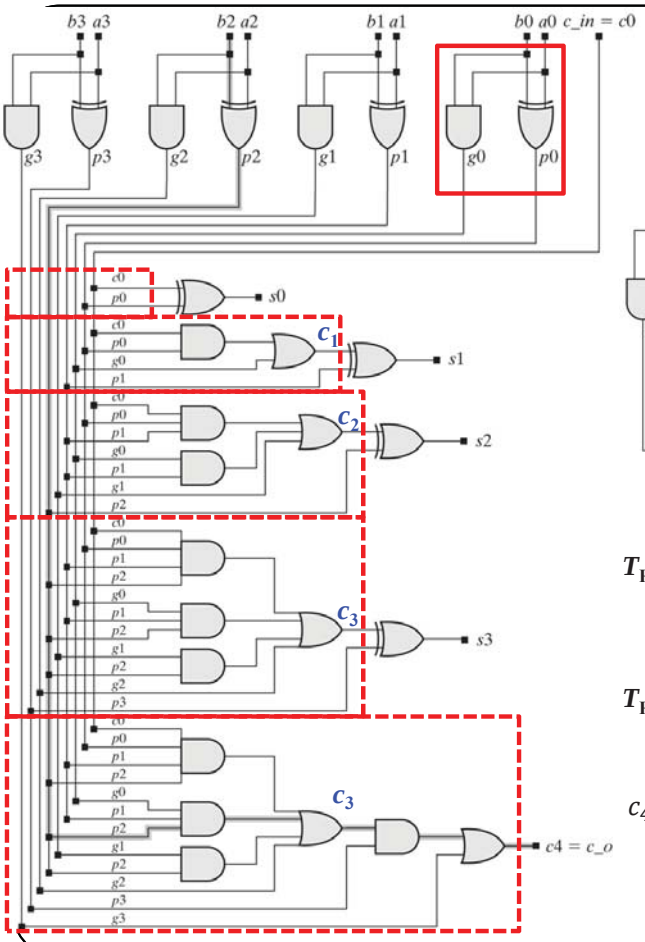
- For k=4, the logic expressions are given as

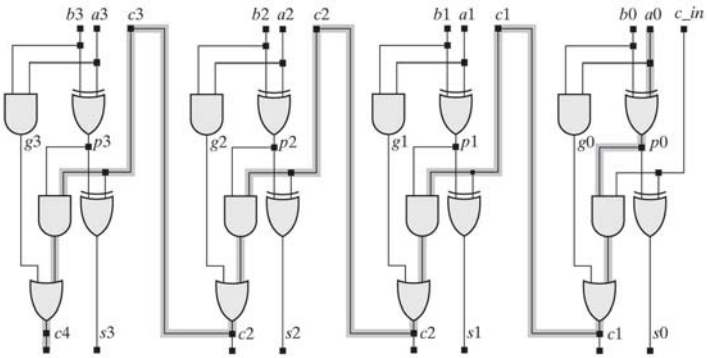
$$c_1 = g_0 + c_0 p_0$$
$$c_2 = g_1 + g_0 p_1 + c_0 p_0 p_1$$
$$c_3 = g_2 + g_1 p_2 + g_0 p_1 p_2 + c_0 p_0 p_1 p_2$$
$$c_4 = g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3 + c_0 p_0 p_1 p_2 p_3 = g_3 + c_3 p_3$$

- Full carry lookahead adder
  - $c_4 = g_3 + c_3 p_3$
  - Impractical for wide-word
    - Penalty for AND and OR networks
    - Increase latency and cost
- Delay time
  - $T_{FCLA}(x,y \rightarrow s) = 2t_{xor} + t_{and} + t_{or}$
  - $T_{FCLA}(x,y \rightarrow c_{out}) = t_{xor} + 2t_{and} + 2t_{or}$



Critical Path Comparison between CLA and RCA





$$T_{RCA}(x_0,y_0 \rightarrow s_3) = T_{FA}(x_i,y_i \rightarrow c_{out}) + 2T_{FA}(c_i \rightarrow c_{out}) + T_{FA}(c_{in} \rightarrow s_i)$$
$$= (t_{xor}+t_{and}+t_{or}) + 2(t_{and}+t_{or}) + t_{xor}$$
$$= 2t_{xor}+3t_{and}+3t_{or}$$

$$T_{RCA}(x_0,y_0 \rightarrow c_4) = T_{FA}(x_i,y_i \rightarrow c_{out}) + 2T_{FA}(c_i \rightarrow c_{out}) + T_{FA}(c_{in} \rightarrow c_{out})$$
$$= (t_{xor}+t_{and}+t_{or}) + 3(t_{and}+t_{or})$$
$$= t_{xor}+4t_{and}+4t_{or}$$

$$c_4 = g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3 + c_0 p_0 p_1 p_2 p_3$$
$$= g_3 + p_3(g_2 + g_1 p_2 + g_0 p_1 p_2 + c_0 p_0 p_1 p_2)$$

$$T_{FCLA}(x_0,y_0 \rightarrow s) = 2t_{xor} + t_{and} + t_{or}$$
$$T_{FCLA}(x_0,y_0 \rightarrow c_{out}) = t_{xor} + 2t_{and} + 2t_{or}$$

Carry Lookahead Beyond 4-Bit

- A 32-bit adder ($k = 32$)

No circuit sharing:
Repeated computations

$$c_1 = g_0 + c_0 p_0$$
$$c_2 = g_1 + g_0 p_1 + c_0 p_0 p_1$$
$$c_3 = g_2 + g_1 p_2 + g_0 p_1 p_2 + \boxed{c_0 p_0 p_1 p_2}$$
$$\vdots$$
$$c_{31} = \underbrace{g_{30} + g_{29} p_{30} + g_{28} p_{29} p_{30} + g_{27} p_{28} p_{29} p_{30} + \ldots + \boxed{c_0 p_0 p_1 p_2}}_{\text{32-input OR}} \underbrace{p_3 \cdots p_{29} p_{30}}_{\text{32-input AND}}$$

  - Require a high fan-in tree-structured circuit

Carry-Lookahead Adder Design

- Efficient approaches of CLA for the high fan-in problem
  - High-radix addition (eg., radix-2^h)
  - Multilevel lookahead
- Radix-16 addition of two binary numbers
  - Two-level CLA with four 4-bit blocks (group)
    - Block generate and block propagate
  - Block generate (i is bit position, where i=4n, and n=0, 1, 2,….)

$$g_{[i,i+3]} = g_{i+3} + g_{i+2} p_{i+3} + g_{i+1} p_{i+2} p_{i+3} + g_i \; p_{i+1} p_{i+2} p_{i+3}$$

  - Block propagate

$$p_{[i,i+3]} = p_i \; p_{i+1} p_{i+2} p_{i+3}$$

$$c_1 = g_0 + c_0 p_0$$
$$c_2 = g_1 + g_0 p_1 + c_0 p_0 p_1$$
$$c_3 = g_2 + g_1 p_2 + g_0 p_1 p_2 + c_0 p_0 p_1 p_2$$
$$c_4 = g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3 + c_0 p_0 p_1 p_2 p_3$$

Carry-Lookahead Adder Design (cont'd)

- Interpretation of the block CLA
  - The four bits positions collectively propagate an incoming carry c_i iff each of the four positions propagates.
  - It collectively generates a carry if a carry is produced in position i+3, or it is produced in position i+2 and propagated by position i+3, …, etc.
- For example i=0, we have

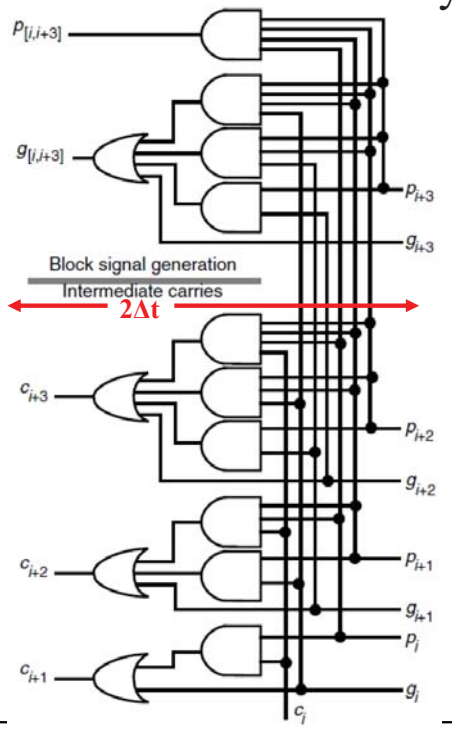
$$g_{[0,3]} = g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3$$

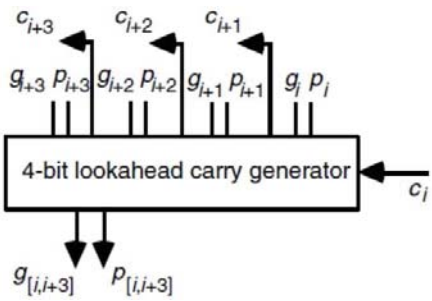
$$p_{[0,3]} = p_0 p_1 p_2 p_3$$

$$\boxed{\begin{aligned} c_1 &= g_0 + c_0 p_0 \\ c_2 &= g_1 + g_0 p_1 + c_0 p_0 p_1 \\ c_3 &= g_2 + g_1 p_2 + g_0 p_1 p_2 + c_0 p_0 p_1 p_2 \\ c_4 &= g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3 + c_0 p_0 p_1 p_2 p_3 \end{aligned}}$$

Carry-Lookahead Adder Design (cont'd)

- A 4-bit lookahead carry generator





$$g_{[0,3]} = g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3$$

$$p_{[0,3]} = p_0 p_1 p_2 p_3$$

$$c_{i+1} = g_i + c_i p_i; \qquad c_i = g_{i-1} + c_{i-1} p_{i-1}$$

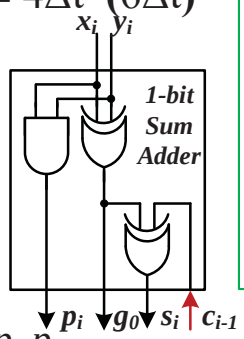
$$\begin{aligned} &= g_i + (g_{i-1} + c_{i-1} p_{i-1}) p_i \\ &= g_i + g_{i-1} p_i + c_{i-1} p_{i-1} p_i \\ &= g_i + g_{i-1} p_i + (g_{i-2} + c_{i-2} p_{i-2}) p_{i-1} p_i \\ &= g_i + g_{i-1} p_i + g_{i-2} p_{i-1} p_i + c_{i-2} p_{i-2} p_{i-1} p_i = \cdots \end{aligned}$$

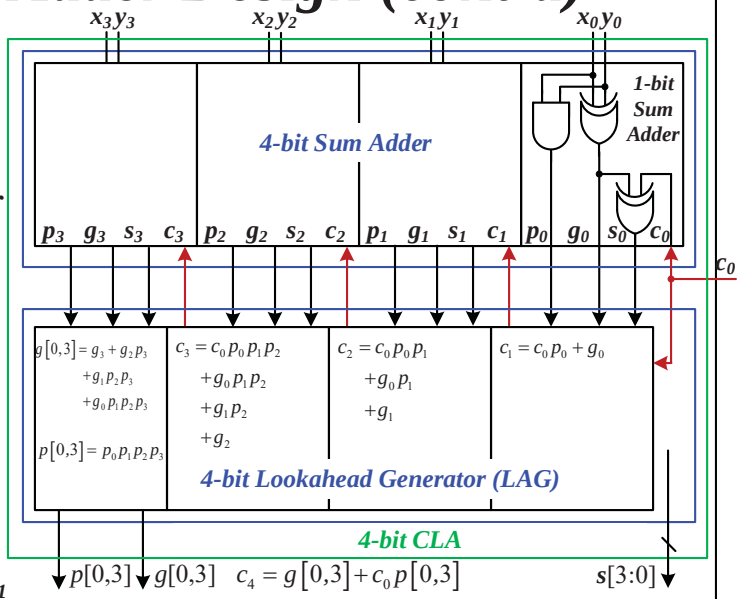
$$\begin{aligned} c_4 &= g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3 + c_0 p_0 p_1 p_2 p_3 \\ &= g_3 + p_3 (g_2 + g_1 p_2 + g_0 p_1 p_2) + c_0 p_0 p_1 p_2 p_3 \\ &= g_{[0,3]} + c_0\, p_{[0,3]} \end{aligned}$$

Δt: Gate delay of AND- or OR-gate)

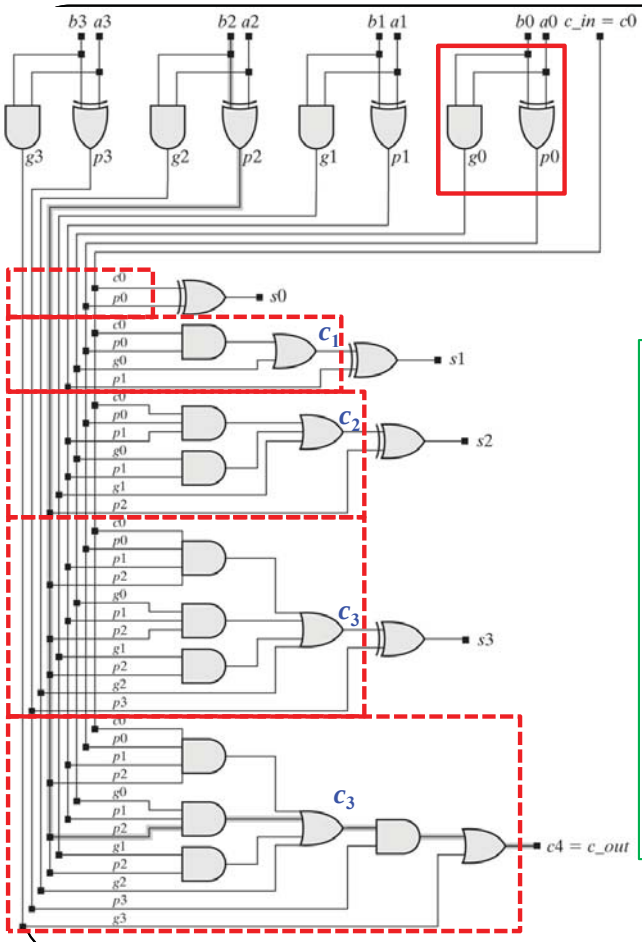
Carry-Lookahead Adder Design (cont'd)

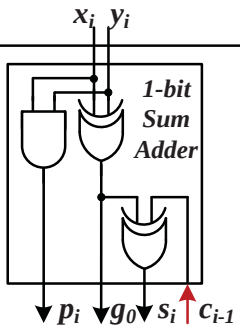
- 4-bit CLA
  - 4-bit sum adder
  - 4-bit lookahead generator
  - Delay time= 4Δt (6Δt)

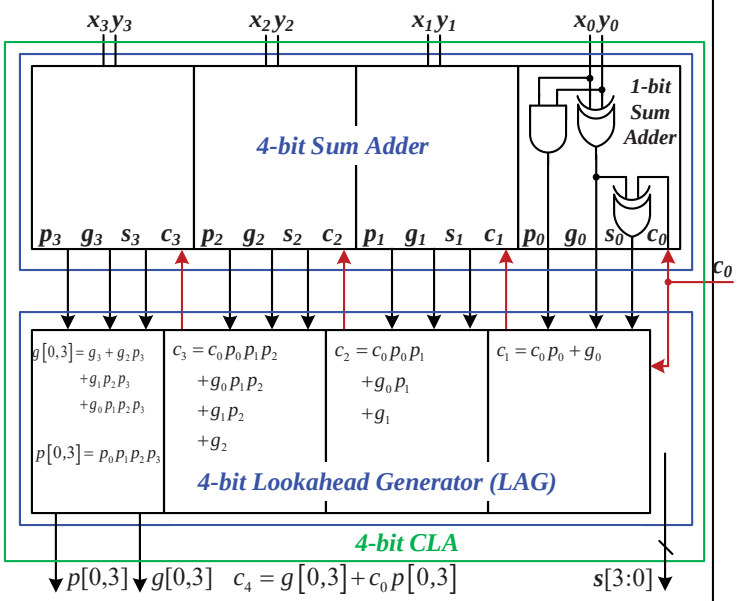


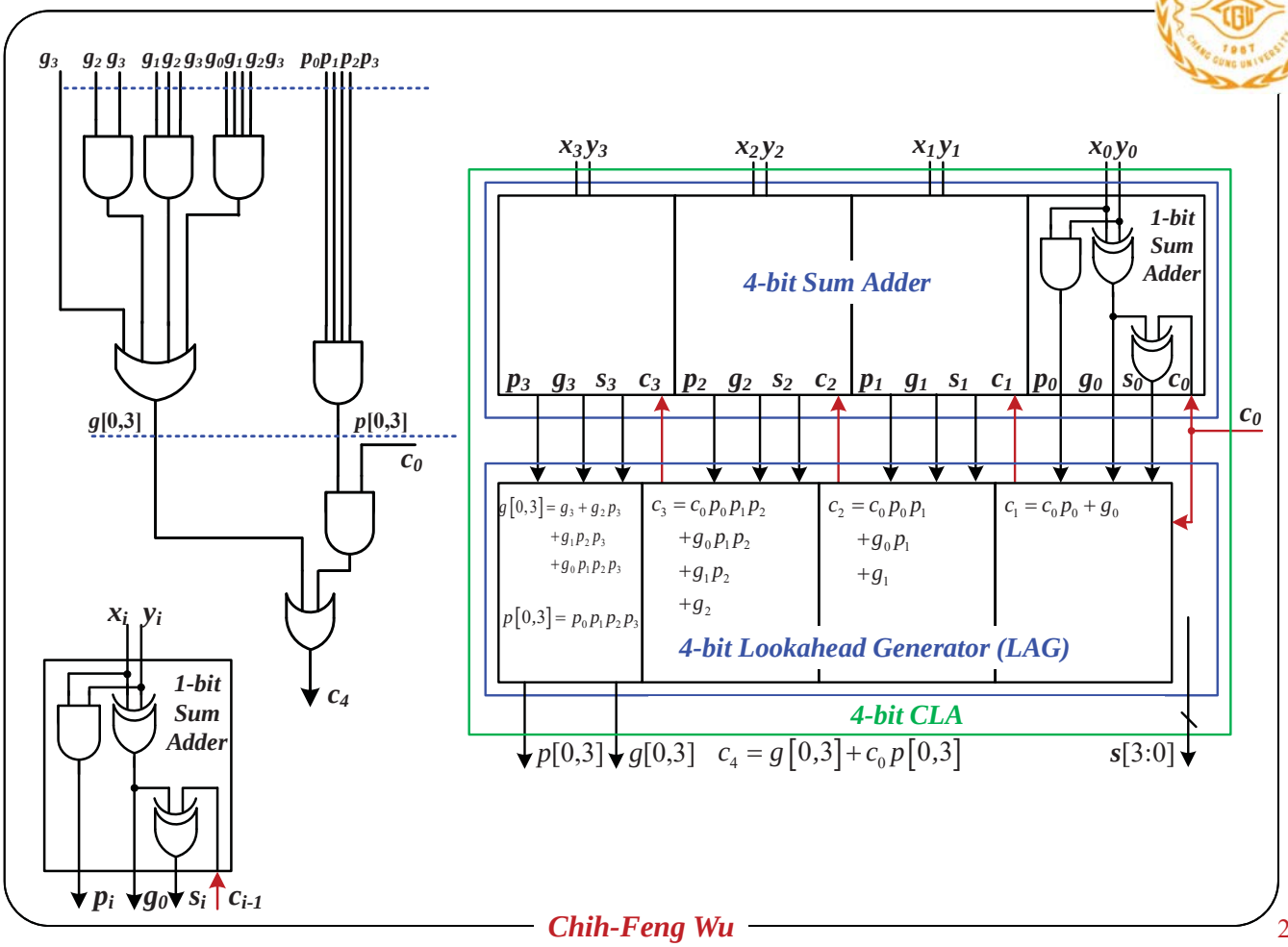


$$c_1 = g_0 + c_0 p_0$$
$$c_2 = g_1 + g_0 p_1 + c_0 p_0 p_1$$
$$c_3 = g_2 + g_1 p_2 + g_0 p_1 p_2 + c_0 p_0 p_1 p_2$$
$$g_{[0,3]} = g_3 + g_2 p_3 + g_1 p_2 p_3 + g_0 p_1 p_2 p_3$$
$$p_{[0,3]} = p_0 p_1 p_2 p_3$$









Multi-Level Carry-Lookahead Adder

- Multi-level architecture to build a carry network of any desired width ($i_0 < i_1 < i_2$)
  - Block generate:

$$g_{[i_0,i_2-1]} = g_{[i_1,i_2-1]} + g_{[i_0,i_1-1]}p_{[i_1,i_2-1]}$$

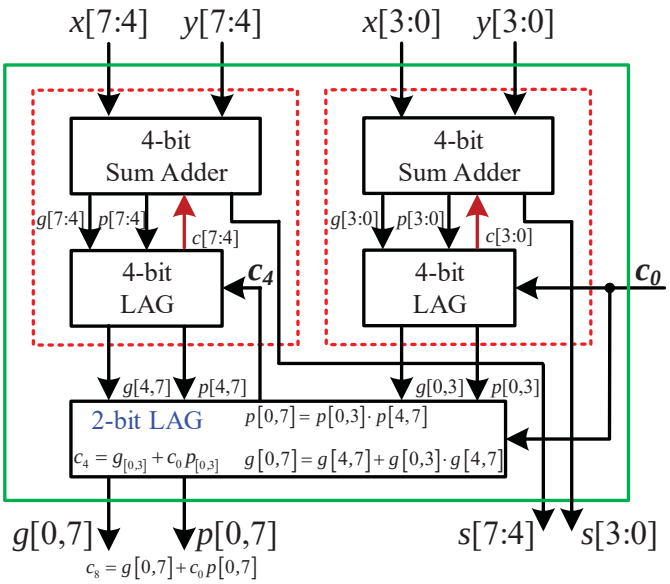
    - A carry generated by the block of positions from i_0 to $i_2 - 1$ iff a carry generated by the $[i_1, i_2 - 1]$ block or a carry generated by the $[i_0, i_1 - 1]$ block propagated by the $[i_1, i_2 - 1]$ block
  - Block propagate

$$p_{[i_0,i_2-1]} = p_{[i_0,i_1-1]}p_{[i_1,i_2-1]}$$

  - In fact, the two blocks can be merged into a large block without contiguous. The can be overlapping.

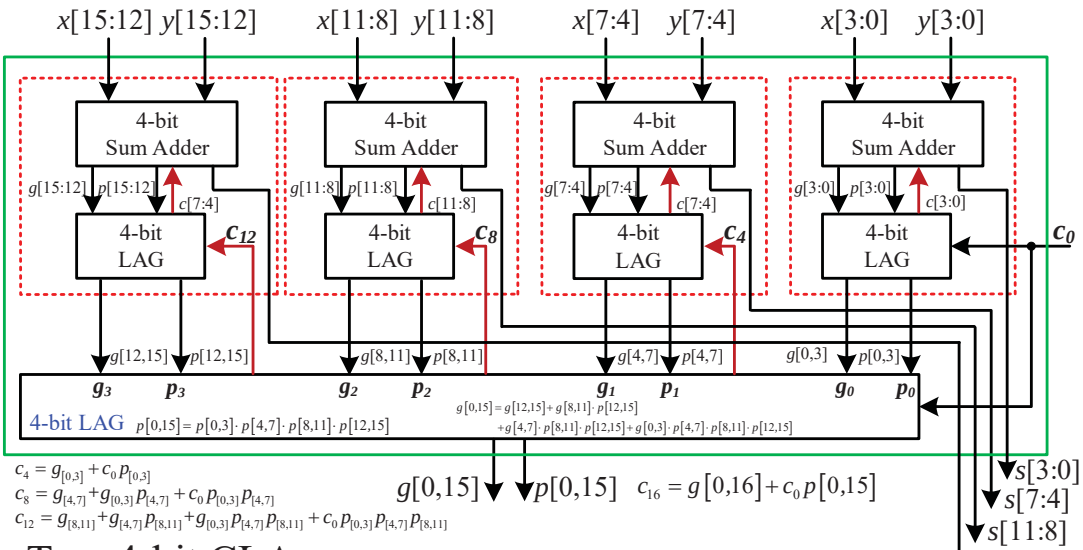
Multi-Level Carry-Lookahead Adder (cont'd)

- 8-bit CLA
  - Two 4-bit sum adders
  - Two 4-bit LAGs
  - One 2-bit LAG
  - Delay time= $7\Delta t$



Multi-Level Carry-Lookahead Adder (cont'd)

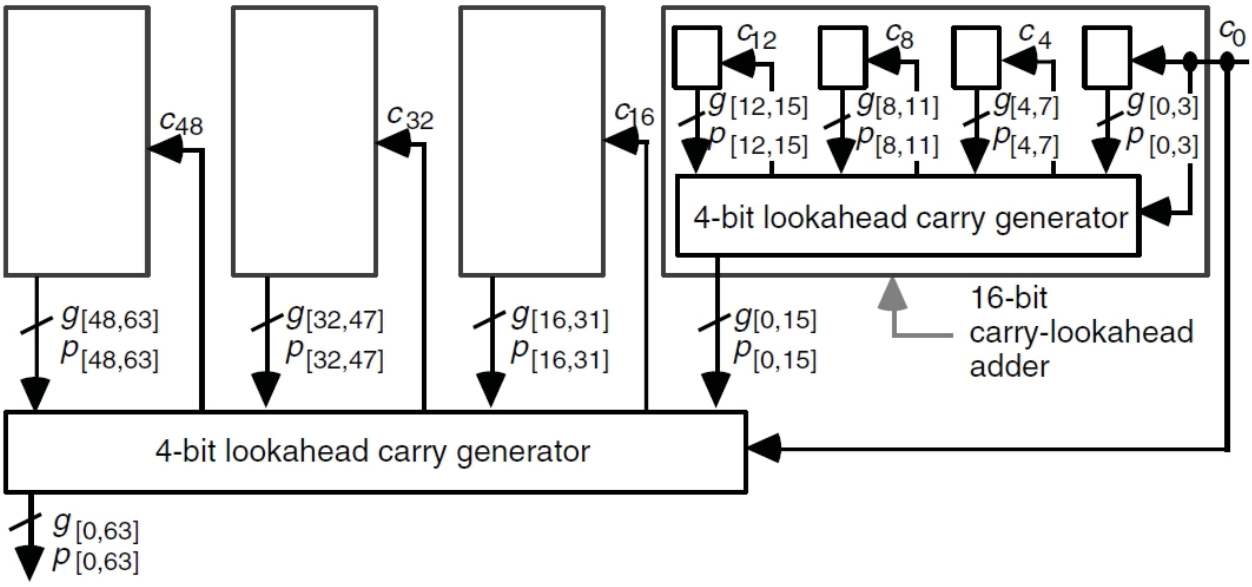
- 16-bit CLA adder



- Two 4-bit CLAs(=Four 4-bit sum adders+ Four 4-bit LAGs)
- One 4-bit LAB
- Delay time = $7\Delta t$

Multi-Level Carry-Lookahead Adder (cont'd)

- 64-bit CLA



  - Total delay = $4log_4k$ gate-level delay
  - Total delay = $4log_4 64 = 12$ gate-level delay

Multi-Level Carry-Lookahead Adder (cont'd)

- 16-bit CLA adder
  - Four 4-bit adders and a 4-bit lookahead carry generator
    - **1** gate-level delay: producing the ***g*** and ***p*** signals from individual bit position
    - **2** gate-level delays: producing the ***g*** and ***p*** signals from 4-bit block
    - **2** gate-level delays: predicting the carry-in, c_4, c_8, and c_{12}
    - **2** gate-level delays: predicting the internal carries within each 4-bit block
    - **1** gate-level delays: computing the sum bits
  - Total delay = 8 gate-level delays
- 16-bit RCA
  - Total delay = 32 gate-level delays (see page-19)

Multi-Level Carry-Lookahead Adder (cont'd)

- Two ways to remedy the carry-out signals c_{64} in 64-bit CLA
  - Approach-1: Generate c_{out} externally based on auxiliary signals or the operand and sum bits in position $k-1$

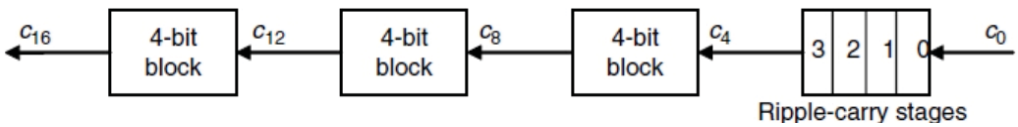
$$c_{out} = g_{[0,k-1]} + c_0 p_{[0,k-1]} = x_{k-1}y_{k-1} + \bar{s}_{k-1}(x_{k-1} + y_{k-1})$$

$$c_{64} = g_{[0,63]} + c_0 p_{[0,63]} = x_{63}y_{63} + \bar{s}_{63}(x_{63} + y_{63})$$

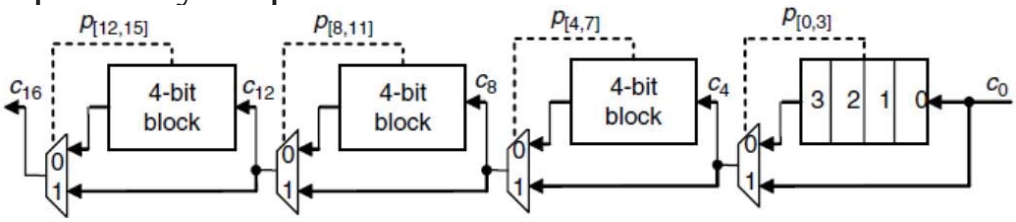
  - Approach-2: Design the adder to be 1-bit wider than needed
    - 65-bit instead of 64-bit using the additional sum bit as c_{out}
- Reduce the number of lookahead levels for a given word width
  - 6-bit or 8-bit lookahead blocks
    - The longer delays introduced by gates with higher fan-in

Simple Carry-Skip Adder

- Carry ripple adder



  - Carry propagation from c_0 to c_{16}
- Simple carry-skip adder

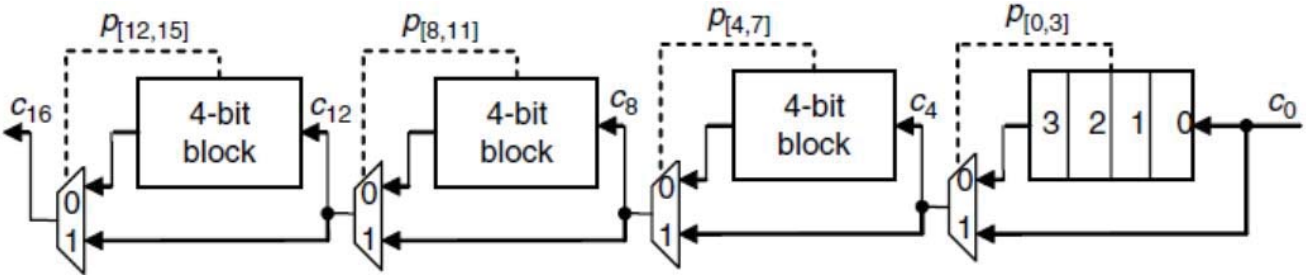


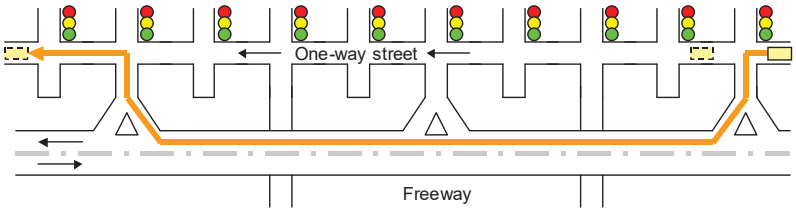
  - In order to speed up carry propagation
    - Establish bypass or skip paths around 4-bit block
  - Group propagate

$p_{[i,i+3]} = p_i\, p_{i+1}\, p_{i+2}\, p_{i+3}$ (4 input AND gate)

Simple Carry-Skip Adder (cont'd)

- Carry skip adder

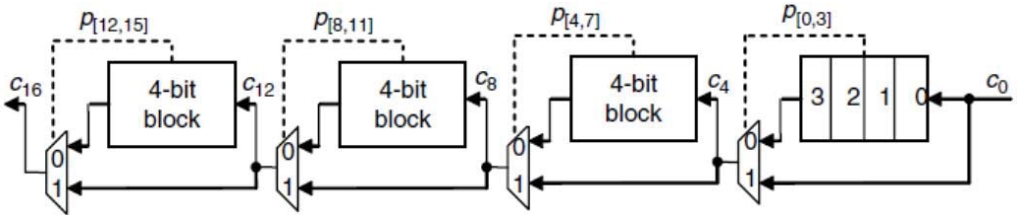




- Street/freeway analogy for carry-skip adder

Simple Carry-Skip Adder (cont'd)

- For example:
  - A carry generated in stage 0, ripples through stages 1-3, goes through the mux., skips the middle two groups, and ripples in the last group from stage 12 to stage **15**. (Note: $c_0=0$)
  - Carry-skip adder: 9 stages of propagation delay (18 gate levels)
  - Ripple carry adder: 14 stages of propagation delay (28 gate levels)
    - Worst case: 16 stages of propagation delay (32 gate levels)



Simple Carry-Skip Adder (cont'd)

- Propagation delay of carry skip adder
  - Carry-propagate delay in a ***k***-bit carry-skip adder with fixed block width ***b*** can be derived as

$$T_{\text{fixed-skip-add}} = \underbrace{(b-1)}_{\text{in block 0}} + \underbrace{1}_{\text{mux}} + \underbrace{(k/b-2)}_{\text{skips}} + \underbrace{(b-1)}_{\text{in last block}}$$

$$\cong 2b + k/b - 3 \text{ stages}$$

  - The optimal fixed block size can be derived as

$$\frac{dT_{fixed-skip-add}}{db} = 2 - \frac{k}{b^2} = 0 \rightarrow b^{opt} = \sqrt{k/2}$$

  - The adder delay with the optimal block size above is

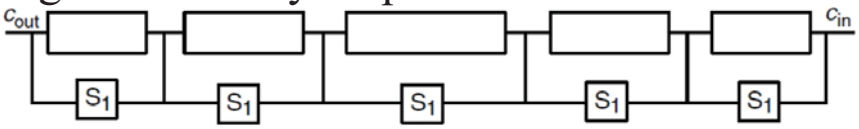
$$T^{opt}_{fixed-skip-add} = 2\sqrt{k/2} + \frac{k}{\sqrt{k/2}} - 3 = 2\sqrt{2k} - 3$$

Simple Carry-Skip Adder (cont'd)

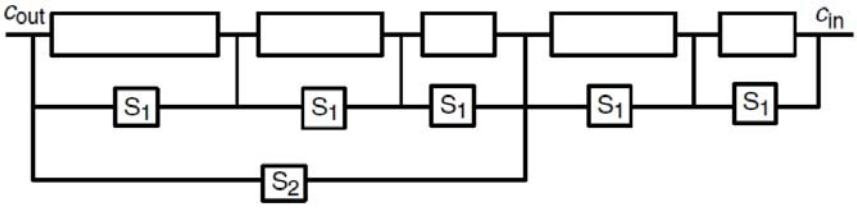
- Example: 32-bit carry-skip adder with fixed-size block
  - $k = 32$, $b^{\text{opt}} = 4$-bit, $T^{\text{opt}} = 13$ stages (26 gate levels)
  - 32-bit ripple carry adder:
    - 32 stages of propagation delay (64 gate levels)
  - $T_{RCA} \approx 2.46\, T^{opt}_{fixed-skip-add}$

Multilevel Carry-Skip Adder

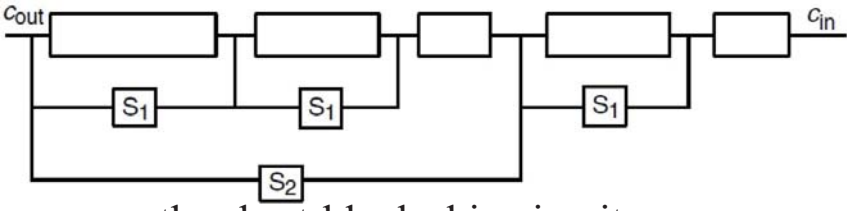
- Single-level carry-skip adder



- Two-level carry-skip adder



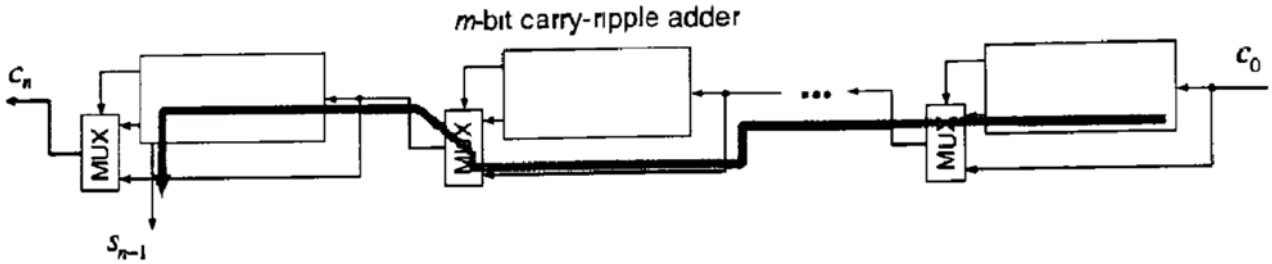
- Optimize two-level carry-skip adder



  - remove the short-block skip circuits

Critical Path Derivation for Carry Skip Adder

- n-bit carry skip adder using (n/m) m-bit carry-ripple adders



  - The delay time of the critical path

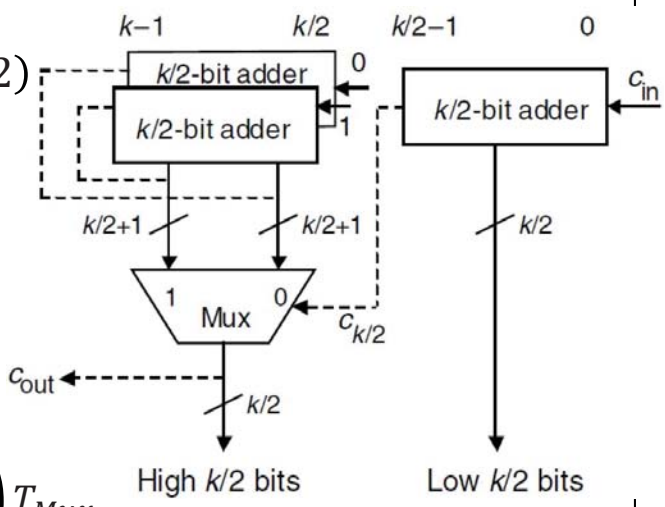
$$T = mt_c + t_{mux} + \left(\frac{n}{m} - 2\right) t_{mux} + (m-1)t_c + t_s$$

$$= (2m-1)t_c + \left(\frac{n}{m} - 1\right) t_{mux} + t_s$$

Carry-Select Adder

- Blocks of bits are added in two ways
  - The correct values (carry-out and sum from $k/2$ to k-1) are selected when $c_{k/2}$ is known (evaluated).
  - k-bit carry-select adder built from 3 m-bit adders ($m = k/2$)
    - Single-level
    - m-bit per group adder
    - **1** m-bit adder for lower half
    - **2** m-bit adders for upper half for $c_{k/2} = 0$ and $c_{k/2} = 1$
  - Delay time

$$T_{Carry-\mathrm{SEL}} = T_{mB\ Adder} + \left(\frac{k}{m} - 1\right) T_{Mux}$$



Carry-Select Adder (cont'd)

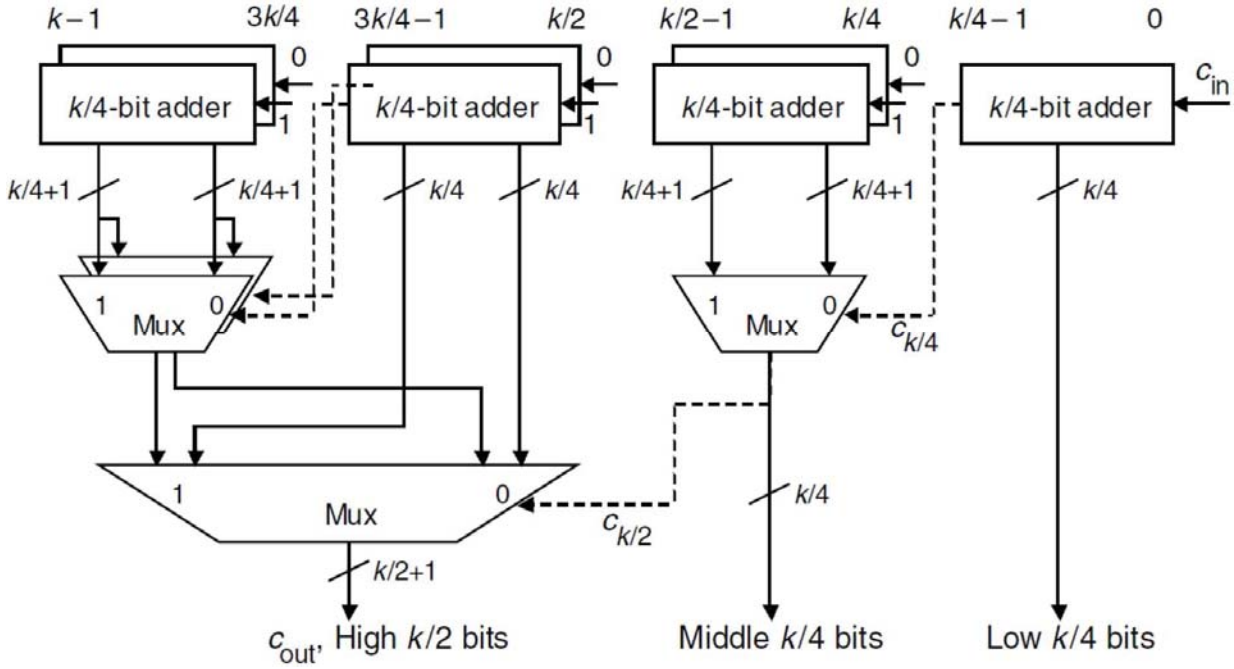
- Cost and delay of a single-bit 2×1 multiplexers
  - $C_{mux} = 1$ and $T_{mux} = 1$
- Cost and delay of a k-bit
  - $C_{add}(k)$ and $T_{add}(k)$
- Cost and delay of a k-bit carry-select adder
  - $C_{carry-select\ add}(k) = 3\ C_{add}(k/2) + \underbrace{k/2 + 1}_{mux}$
  - $T_{carry-select\ add}(k) = T_{add}(k/2) + 1$
- Cost-effective measurement

$$C_{carry-select\ add}(k) T_{carry-select\ add}(k) < C_{add}(k) T_{add}(k)$$

$$[3\ C_{add}(k/2) + k/2 + 1][T_{add}(k/2) + 1] < C_{add}(k) T_{add}(k)$$

Multilevel Carry-Select Adder

- Two-level carry-select adder



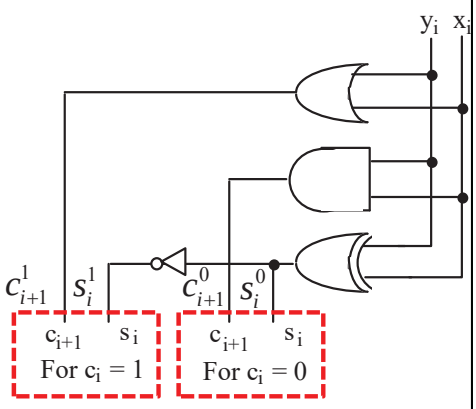
Conditional Sum Adder

- The cost and delay of a conditional-sum adder

  $C(k) \cong 2C(k/2) + k + 2 \cong k(\log_2 k + 2) + k\,C(1)$

  $T(k) = T(k/2) + 1 = \log_2 k + T(1)$

  - Both $C(1)$ and $T(1)$ are the cost and delay of the circuit of **1-bit addition in a conditional-sum adder**.
- Basic design principle
  - Step-1: Generate all possible sum and carry bits in a single step
  - Step-2: Use the carry bits to select the correct sum bits
  - $(c_{i+1}, s_i) = \begin{cases} (c_{i+1}^0, s_i^0), & if\ c_i = 0 \\ (c_{i+1}^1, s_i^1), & if\ c_i = 1 \end{cases}$



Conditional Sum Adder (cont'd)

- Operating procedure
  - Step-1: Generate both sum and carry bits for two cases, with carry-in being $c_{in} = 0$ and $c_{in} = 1$, in each bit position
  - Step-2: The stages (bit-position) are paired and the possible carriers between the two stages (bit-position) in a pair are considered.
  - Step-3: Produce both sum and carry bits for pairs of stage
- Complexity
  - The number of decision steps (selection levels) required for N-bit addition with m-bit group: $log_2 \frac{N}{m}$

Conditional Sum Addition of Two 8-Bit Numbers

- $N = 8$ and $m = 1$
- Number of 2-to-1 Muxs

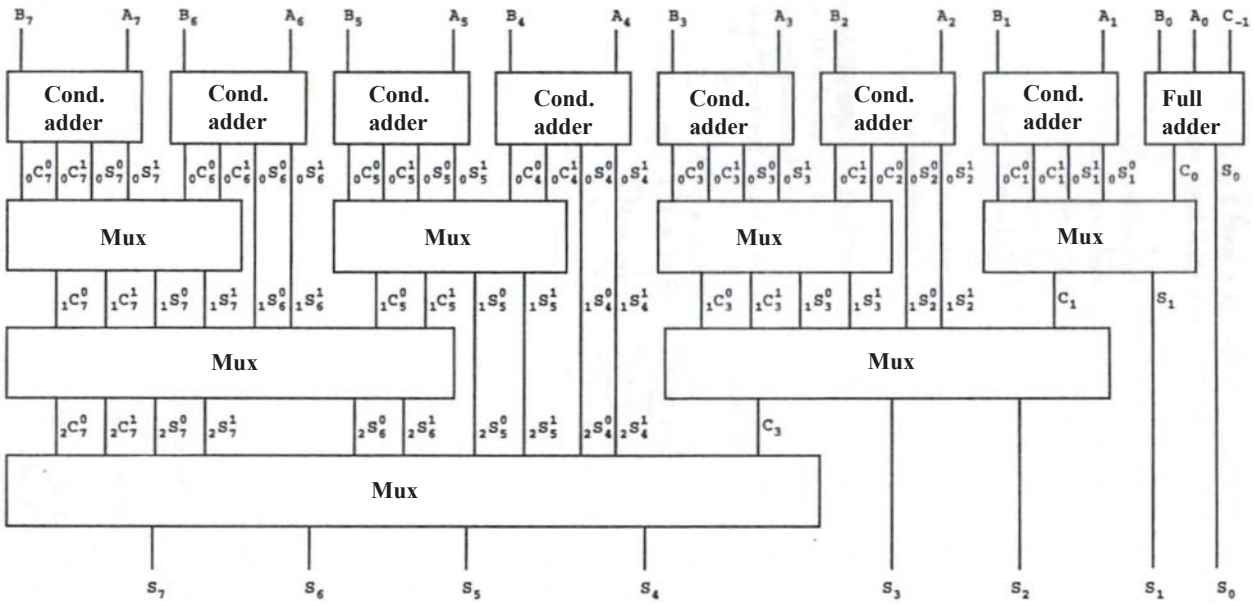
$$\left(\frac{N}{2}+1\right) + 3\left(\frac{N}{4}+1\right) + 7\left(\frac{N}{8}+1\right) + \cdots + (N-1)2$$
$$= (N-1)(log_2 N + 1)$$

- Delay time

$$T_{cond_sum} = t_{add-m} + log_2\left(\frac{N}{m}\right) t_{Mux}$$

	7	6	5	4	3	2	1	0	
x	0	1	0	1	1	0	1	1	$c_0 = 0$
y	0	1	0	1	0	1	0	1	
S^0	0	0	0	0	1	1	1	0	Step 1
c^0	0	1	0	1	0	0	0	1	
S^1	1	1	1	1	0	0	0		
c^1	0	1	0	1	1	1	1		
S^0	1	0	1	0	1	1	0	0	Step 2
c^0	0		0		0		1		
S^1	1	1	1	1	0	0			
c^1	0		0		1				
S^0	1	0	1	0	0	0	0	0	Step 3
c^0	0				1				
S^1	1	0	1	1					
c^1	0								
S	1	0	1	1	0	0	0	0	

Conditional Sum Addition of Two 8-Bit Numbers (cont'd)



Conditional-Sum Addition of Two 16-bit Numbers

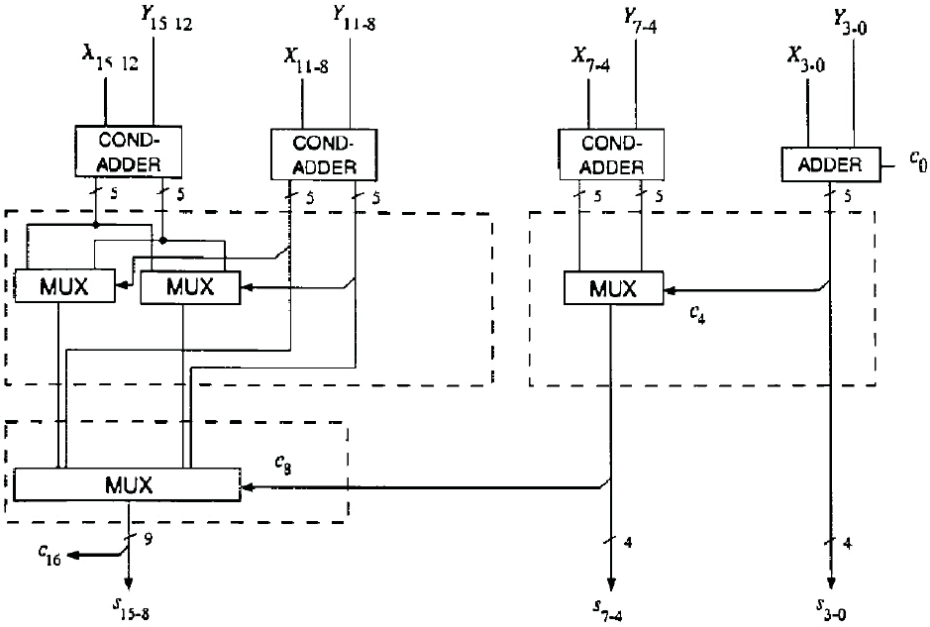
Block width	Block carry-in		15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
		x	0	0	1	0	0	1	1	0	1	1	1	0	1	0	1	0
		y	0	1	0	0	1	0	1	1	0	1	0	1	1	1	0	1
			Block sum and block carry-out															
1	0	s	0	1	1	0	1	1	0	1	1	0	1	1	0	1	1	1
		c	0	0	0	0	0	0	1	0	0	1	0	0	1	0	0	0
	1	s	1	0	0	1	0	0	1	0	0	1	0	0	1	0	0	
		c	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	
2	0	s	0	1	1	0	1	1	0	1	0	0	1	1	0	1	1	1
		c	0		0		0		1		1		0		1		0	
	1	s	1	0	1	1	0	0	1	0	0	1	0	0	1	0		
		c	0		0		1		1		1		1		1			
4	0	s	0	1	1	0	0	0	0	1	0	0	1	1	0	1	1	1
		c	0				1				1				1			
	1	s	0	1	1	1	0	0	1	0	0	1	0	0				
		c	0				1				1							
8	0	s	0	1	1	1	0	0	0	1	0	1	0	0	0	1	1	1
		c	0								1							
	1	s	0	1	1	1	0	0	1	0								
		c	0															
16	0	s	0	1	1	1	0	0	1	0	0	1	0	0	0	1	1	1
		c	0															
	1	s																
		c																

c_{in} 0

c_{out}

Architecture of Conditional Sum Adder

- 16-bit conditional sum adder using 4-bit adder and multiplexer
  - $N = 16$
  - $m = 4$

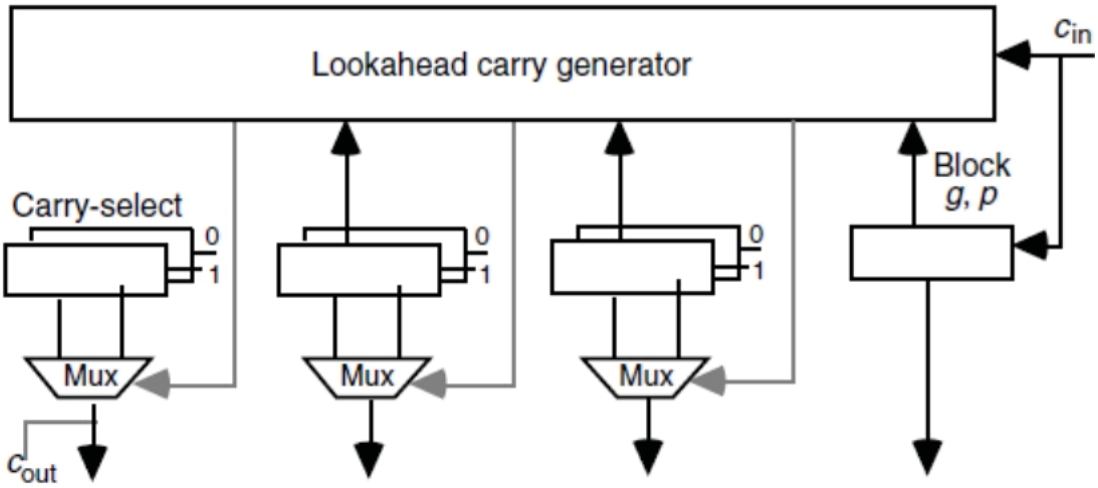


- Delay time

$T_{Cond-Sum} = T_{4B\ Adder} + log_2(N/m)T_{Mux} = T_{4B\ Adder} + 2T_{Mux}$

Hybrid Design and Optimization

- Hybrid carry-lookahead/carry-select adder



Modular Two-Operand Adder

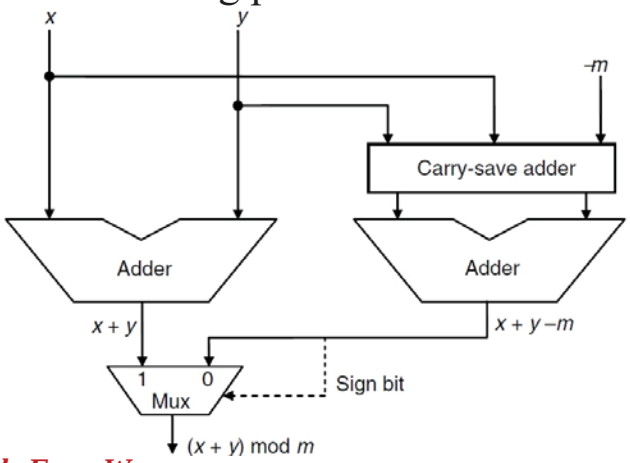
- The sum of input operands ***x*** and ***y*** need to compute modulo given a constant ***m***.
  - $(x + y) \bmod m$
- For a general modulus ***m***
  - Compare and calculate as the following pseudo-cod
  - Pseudo code

    if $(x + y) \geq m$

    $\quad x + y - m$

    else

    $\quad x + y$



Modular Two-Operand Adder (cont'd)

- Consideration of a simple modulo-***m*** operations
  - $m = 2^k$: ignore c_{out} at k-bit position
  - $m = 2^k - 1$: use end-around carry because $2^k = (2^k - 1) + 1$
  - $m = 2^k + 1$: need to $(k - 1)$-bit express residue

Modular Two-Operand Adder (cont'd)

- Diminished-1 encoding for $m = 2^k + 1$
- For example 7-3: Design a modulo-17 adder for unsigned integer operands in the range [0, 16] represented in the diminished-1 format.
  - $x \neq 0, y \neq 0, x + y \neq m = 17$
  - $(x + y)$ **mod** $17 = \begin{cases} x + y - 1, & x + y \leq 17 \\ x + y - 17, & x + y \geq 18 \end{cases}$
- Revision of above formulation as below
  - $(x + y)$ **mod** 17
    $= \begin{cases} (x-1) + (y-1) + 1, & (x-1) + (y-1) \leq 15 \\ (x-1) + (y-1) - 16, & (x-1) + (y-1) \geq 16 \end{cases}$
  - $(x - 1) + (y - 1)$ with an inverted end-around carry generating the desired result

Modular Two-Operand Adder (cont'd)

- $(x + y)$ **mod** 17
  $= \begin{cases} (x-1) + (y-1) + 1, & (x-1) + (y-1) \leq 15 \\ (x-1) + (y-1) - 16, & (x-1) + (y-1) \geq 16 \end{cases}$

- End-around policy

1. Inverted end-around carry in either case.
   - Add 1 to the sum of $(x - 1)$ and $(y - 1)$ when $c_{out} = 0$, i.e., $(x-1) + (y-1) \leq 15$
   - Subtract 16 (by dropping the outgoing carrier) when $c_{out} = 1$, i.e.,$(x-1) + (y-1) \geq 16$

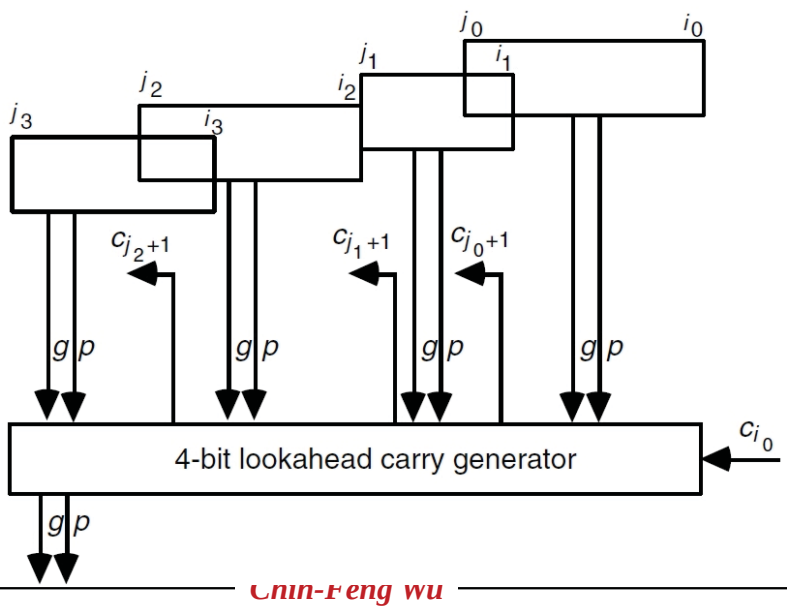
Multi-Level Carry-Lookahead Adder (cont'd)

- An overlapping block $[i_1, j_1]$ and $[i_0, j_0] i_0 \leq i_1 - 1 \leq j_0 < j_1$
  - Block generate:

$$g_{[i_0,j_1]} = g_{[i_1,j_1]} + g_{[i_0,i_0]} p_{[i_1,j_1]}$$

  - Block propagate

$$p_{[i_0,j_1]} = p_{[i_0,j_0]} p_{[i_1,i_1]}$$

Multi-Level Carry-Lookahead Adder (cont'd)

- A 4-bit lookahead carry generator
  - Combine the ***g*** and ***p*** signals from adjacent or overlapping blocks into the ***p*** and ***g*** signals for the combined block